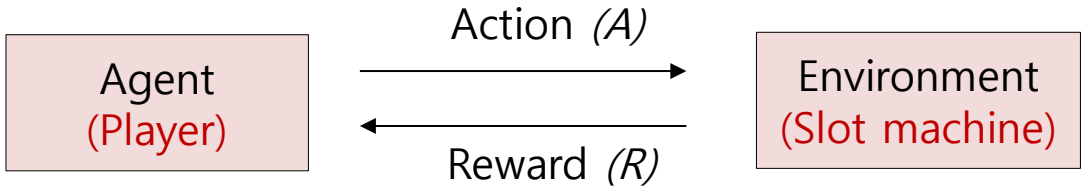
Bandit Problem

Prof. Tae-Hyoung Park
Dept. of Intelligent Systems & Robotics, CBNU

Bandit Problem

- Bandit
  - Bandit = Slot machine
  - Multi-armed bandit problem
    - n 개 Slot machine 의 특성 (승률) 이 각각 다름
    - Player 는 정해진 횟수 (예: 1000 회) 를 플레이함
    - Player 는 처음에는 slot machine 에 대한 정보를 알지 못함. 플레이를 해보고 결과를 확인하면서 좋은 slot machine 을 찾아내야 함
    - 정해진 횟수안에 최대의 coin 을 얻는 것을 목표로 함



- ✓ Action : 하나의 slot machine 을 선택해서 play
- ✓ Reward : coins
- ✓ State : 정의되지 않음

Action Value

- Action Value (행동 가치)
  - Probability distribution of slot machine (확률분포)

슬롯머신 a

얻을 수 있는 동전 개수	0	1	5	10
확률	0.70	0.15	0.12	0.03

슬롯머신 b

얻을 수 있는 동전 개수	0	1	5	10
확률	0.50	0.40	0.09	0.01

  - Expectation value of slot machine (기댓값)
    - **슬롯머신 a**: $(0 \times 0.70) + (1 \times 0.15) + (5 \times 0.12) + (10 \times 0.03) = \mathbf{1.05}$
    - **슬롯머신 b**: $(0 \times 0.50) + (1 \times 0.40) + (5 \times 0.09) + (10 \times 0.01) = \mathbf{0.95}$
  - Action value (or Value)
    - Expectation value of reward resulted from an action

(ex) 슬롯머신 a 를 선택하여 얻는 코인 개수의 기댓값

$$Q(a) = E(R \mid A = a) \quad \text{or} \quad Q(a) = E(R \mid a)$$

Action Value

- Estimation of Action Value

(ex) 두 대의 slot machine 으로 한번 play 하여 얻은 결과

슬롯머신	결과
	첫 번째
a	0
b	1

$$Q(a) = 0$$
$$Q(b) = 1$$

(ex) 두 대의 slot machine 으로 세번 play 하여 얻은 결과

슬롯머신	결과		
	첫 번째	두 번째	세 번째
a	0	1	5
b	1	0	0

$$Q(a) = \frac{0+1+5}{3} = 2$$
$$Q(b) = \frac{1+0+0}{3} = 0.333$$

Action Value

- Estimation of Action Value
  - 하나의 slot machine 에 대하여

    R_n : n 번째 action 에서의 reward

    Q_n : n 번째 action 에서의 action value 의 추정치

$$\boxed{Q_n = \frac{R_1 + R_2 + \cdots + R_n}{n}}$$

  - Incremental equation

$$Q_n = \frac{R_1 + R_2 + \cdots + R_n}{n}$$

$$Q_{n-1} = \frac{R_1 + R_2 + \cdots + R_{n-1}}{n-1}$$

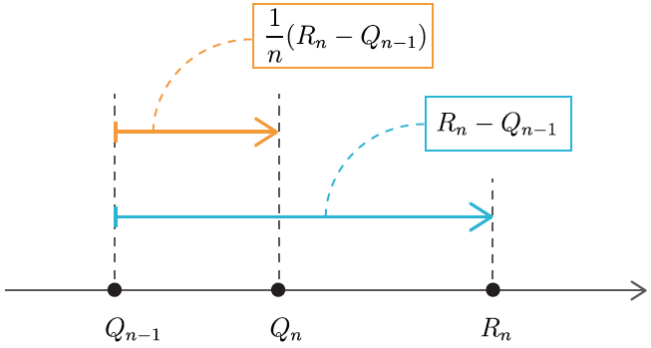
$$= \frac{1}{n}(\underbrace{R_1 + \cdots + R_{n-1}}_{(n-1)Q_{n-1}} + R_n)$$

$$= \frac{1}{n}\{(n-1)Q_{n-1} + R_n\}$$

$$= \left(1 - \frac{1}{n}\right)Q_{n-1} + \frac{1}{n}R_n$$

$$\boxed{= Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1})}$$

$\frac{1}{n}$: *learning rate (학습률)*



$$NewEstimate \leftarrow OldEstimate + StepSize\left[Target - OldEstimate\right]$$

Decision Making

1) Exploitation (활용)
   - Take the most rewarding action given current knowledge
   - Greedy selection
2) Exploration (탐색)
   - Take an action to gather more knowledge
   - Random selection

(Ex)

- Restaurant Selection
  - Exploitation: Go to your favorite restaurant
  - Exploration: Try a new restaurant
- Oil Drilling
  - Exploitation: Drill at the best known location
  - Exploration: Drill at a new location
- Game Playing
  - Exploitation: Play the move you believe is best
  - Exploration: Play an experimental move

Policy of Player

- Bandit problem

1) Exploitation (활용)
   - 지금까지 play 한 결과를 바탕으로 가장 좋아 보이는 slot machine (action) 선택

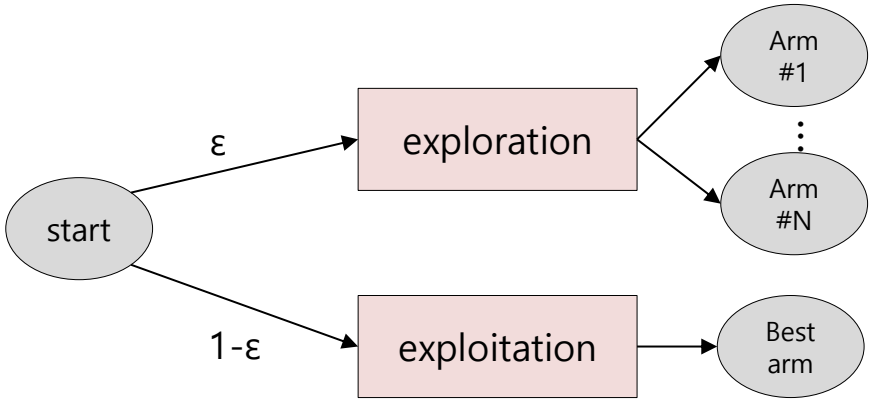
   ☞ 현재의 Value 가 가장 높은 slot machine 선택 : greedy

2) Exploration (탐색)
   - 어떠한 action 이 가치가 있는 지를 정확하게 추정하기 위해 다양한 slot machine (action) 의 선택을 시도함

   ☞ 무작위로 slot machine 을 선택: random

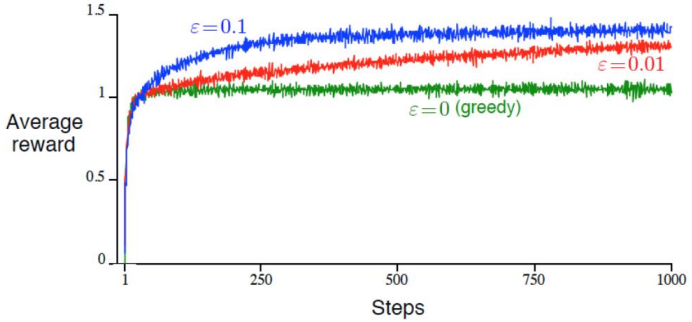
Policy of Player

- ε-greedy
  - exploitation & exploration
  - ε (0≤ε≤1) 의 확률로 exploration , 1- ε 의 확률로 exploitation



$\varepsilon = 0 \rightarrow$ exploitation (greedy)
$\varepsilon = 1 \rightarrow$ exploration (random)

```
def get_action(self):  # 행동 선택(ε-탐욕 정책)
    if np.random.rand() < self.epsilon:
        return np.random.randint(0, len(self.Qs))  # 무작위 행동 선택
    return np.argmax(self.Qs)  # 탐욕 행동 선택
```



Implementation

- Slot Machine

ch01/bandit.py

```python
import numpy as np

class Bandit:
    def __init__(self, arms=10):  # arms = 슬롯머신 대수
        self.rates = np.random.rand(arms)  # 슬롯머신 각각의 승률 설정(무작위)

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

(ex)

```python
bandit = Bandit()

for i in range(3):
    print(bandit.play(0))
```

출력 결과

```
1
0
0
```

Implementation

- Agent

ch01/bandit.py

```
class Agent:
    def __init__(self, epsilon, action_size=10):
        self.epsilon = epsilon  # 무작위로 행동할 확률(탐색 확률)
        self.Qs = np.zeros(action_size) # 각 슬롯머신의 가치 추정치
        self.ns = np.zeros(action_size) # 각 슬롯머신의 플레이 횟수

    def update(self, action, reward):  # 슬롯머신의 가치 추정
        self.ns[action] += 1
        self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]

    def get_action(self):  # 행동 선택(ε-탐욕 정책)
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))  # 무작위 행동 선택
        return np.argmax(self.Qs)  # 탐욕 행동 선택
```

Implementation

실습 #1 bandit.py

```python
import numpy as np
import matplotlib.pyplot as plt


class Bandit:
    def __init__(self, arms=10):  # arms = 슬롯머신 대수
        self.rates = np.random.rand(arms)  # 슬롯머신 각각의 승률 설정(무작위)

    def play(self, arm):
        rate = self.rates[arm]
        if rate > np.random.rand():
            return 1
        else:
            return 0


class Agent:
    def __init__(self, epsilon, action_size=10):
        self.epsilon = epsilon  # 무작위로 행동할 확률(탐색 확률)
        self.Qs = np.zeros(action_size)
        self.ns = np.zeros(action_size)

    # 슬롯머신의 가치 추정
    def update(self, action, reward):
        self.ns[action] += 1
        self.Qs[action] += (reward - self.Qs[action]) / self.ns[action]

    # 행동 선택(ε-탐욕 정책)
    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))  # 무작위 행동 선택
        return np.argmax(self.Qs)  # 탐욕 행동 선택
```

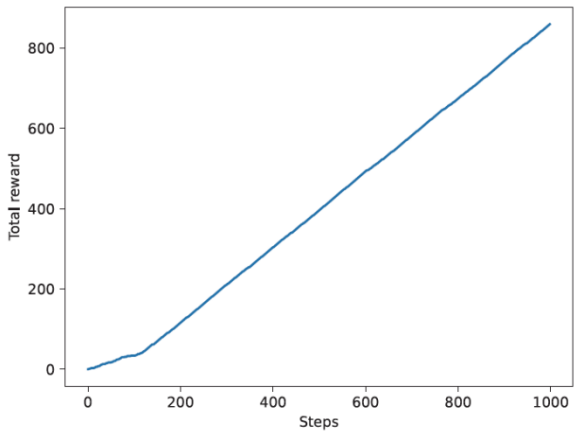
```python
if __name__ == '__main__':
    steps = 1000
    epsilon = 0.1

    bandit = Bandit()
    agent = Agent(epsilon)
    total_reward = 0
    total_rewards = []  # 보상 합
    rates = []          # 승률

    for step in range(steps):
        action = agent.get_action()   # 행동 선택
        reward = bandit.play(action)  # 실제로 플레이하고 보상을 받음
        agent.update(action, reward)  # 행동과 보상을 통해 학습
        total_reward += reward

        total_rewards.append(total_reward)       # 현재까지의 보상 합 저장
        rates.append(total_reward / (step + 1))  # 현재까지의 승률 저장

    print(total_reward)

    # [그림 1-12] 단계별 보상 총합
    plt.ylabel('Total reward')
    plt.xlabel('Steps')
    plt.plot(total_rewards)
    plt.show()

    # [그림 1-13] 단계별 승률
    plt.ylabel('Rates')
    plt.xlabel('Steps')
    plt.plot(rates)
    plt.show()
```

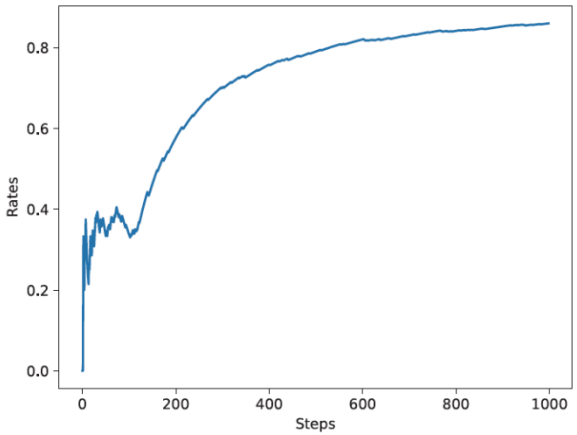
Implementation

실습 #1

출력 결과

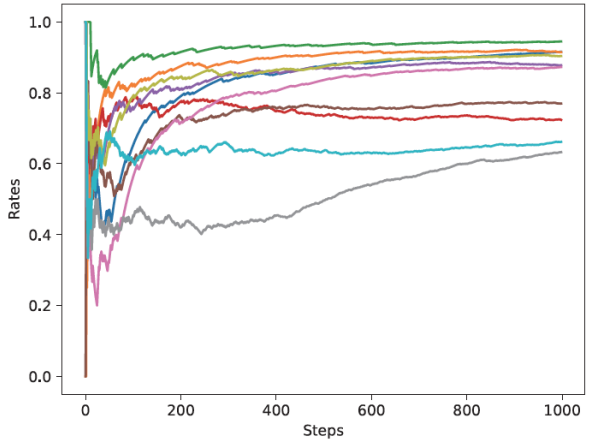
859





- Random number generation → 실행 시 마다 다른 결과

그림 1-14 10번의 결과를 한꺼번에 그려보기



Implementation

실습 #2 bandit_avg.py

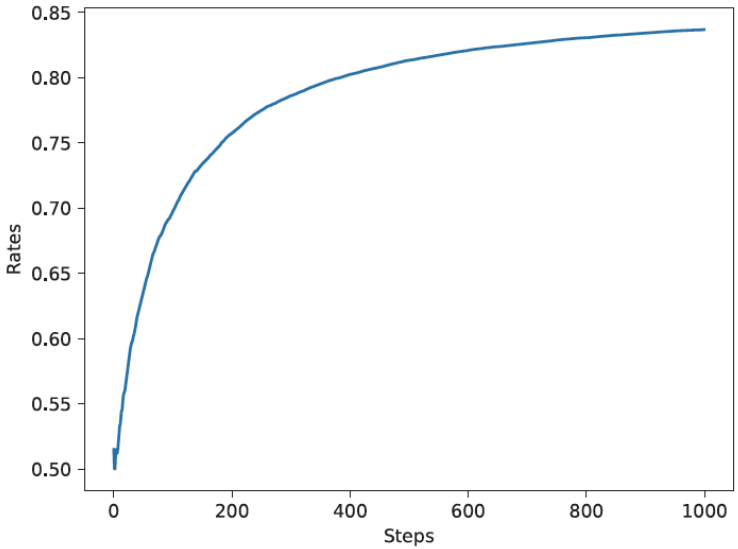
```python
import numpy as np
import matplotlib.pyplot as plt
from bandit import Bandit, Agent


runs = 200
steps = 1000
epsilon = 0.1
all_rates = np.zeros((runs, steps))  # (200, 1000) 형상 배열

for run in range(runs):  # 200번 실험
    bandit = Bandit()
    agent = Agent(epsilon)
    total_reward = 0
    rates = []

    for step in range(steps):
        action = agent.get_action()
        reward = bandit.play(action)
        agent.update(action, reward)
        total_reward += reward
        rates.append(total_reward / (step + 1))

    all_rates[run] = rates  # 보상 결과 기록

avg_rates = np.average(all_rates, axis=0)  # 각 단계의 평균 저장

# [그림 1-16] 단계별 승률(200번 실험 후 평균)
plt.ylabel('Rates')
plt.xlabel('Steps')
plt.plot(avg_rates)
plt.show()
```

	1	2	3	...	1000
1번째 실험	1.0	0.5	0.333	...	0.913
2번째 실험	0.0	0.0	0.0	...	0.821
...	...	...	...	...	...
200번째 실험	1.0	1.0	1.0	...	0.615
평균	0.493	0.497	0.504	...	0.838

Implementation

실습 #2

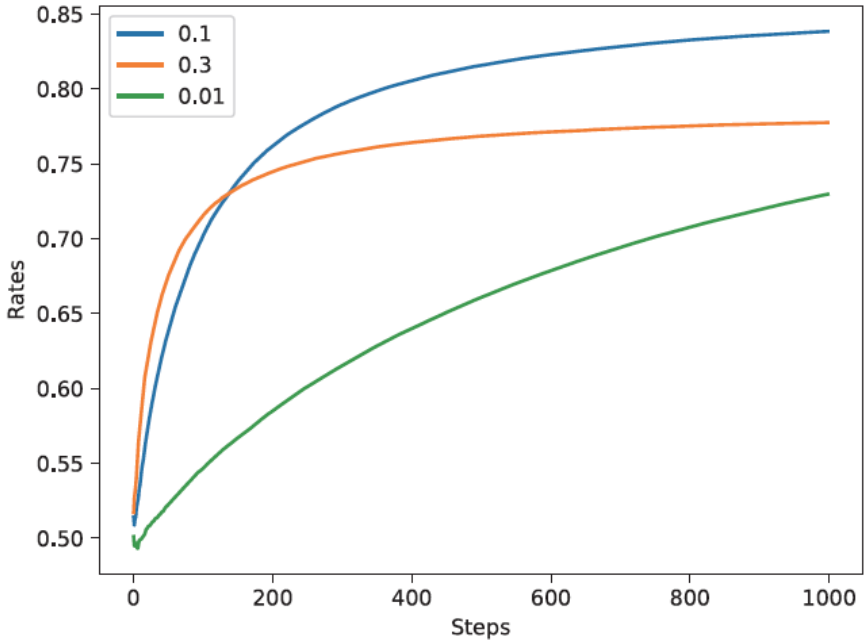
그림 1-16 단계별 승률(200번 실험 후 평균)



Quiz

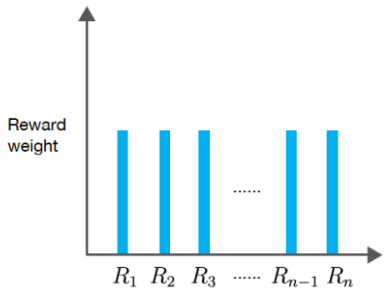
(Q) 다음 그래프를 출력하는 Bandit 프로그램 작성하라

그림 1-17 ε-탐욕 정책의 εε값을 바꾼 결과



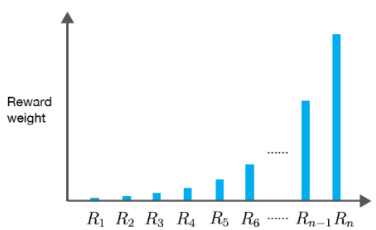
Non-stationary Bandit Problem

- Reward weight
  - Stationary
    - Slot machine 의 승률은 초기 고정값 유지
    - Rewards $R_1, R_2, \cdots, R_n$ 에 동일한 weight (가중치) 부여
    - 새로 얻은 보상과 과거의 보상을 동등하게 취급



$$
\begin{aligned}
Q_n &= \frac{R_1 + R_2 + \cdots + R_n}{n} \\
&= \frac{1}{n}R_1 + \frac{1}{n}R_2 + \cdots + \frac{1}{n}R_n \\
&= Q_{n-1} + \frac{1}{n}(R_n - Q_{n-1})
\end{aligned}
$$

  - Non-stationary
    - Slot machine 의 승률은 매번 변경
    - Rewards $R_1, R_2, \cdots, R_n$ 에 다른 weight (가중치) 부여
    - 새로 얻은 보상에 대하여 과거의 보상보다 높은 가중치 부여



$$
\begin{aligned}
Q_n &= Q_{n-1} + \alpha(R_n - Q_{n-1}) \\
&= \alpha R_n + Q_{n-1} - \alpha Q_{n-1} \\
&= \alpha R_n + (1-\alpha)Q_{n-1}
\end{aligned}
$$

$$
\begin{aligned}
Q_n &= \alpha R_n + (1-\alpha)Q_{n-1} \\
&= \alpha R_n + (1-\alpha)\{\alpha R_{n-1} + (1-\alpha)Q_{n-2}\} \\
&= \alpha R_n + \alpha(1-\alpha)R_{n-1} + (1-\alpha)^2 Q_{n-2}
\end{aligned}
$$

$$
\boxed{Q_n = \alpha R_n + \alpha(1-\alpha)R_{n-1} + \cdots + \alpha(1-\alpha)^{n-1}R_1 + (1-\alpha)^n Q_0}
$$

Implementation

- NonStatBandit

ch01/non_stationary.py

```
class NonStatBandit:
    def __init__(self, arms=10):
        self.arms = arms
        self.rates = np.random.rand(arms)

    def play(self, arm):
        rate = self.rates[arm]
        self.rates += 0.1 * np.random.randn(self.arms)  # 노이즈 추가
        if rate > np.random.rand():
            return 1
        else:
            return 0
```

- AlphaAgent

ch01/non_stationary.py

```
class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions)
        self.alpha = alpha  # 고정값 α

    def update(self, action, reward):
        # α로 갱신
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

Implementation

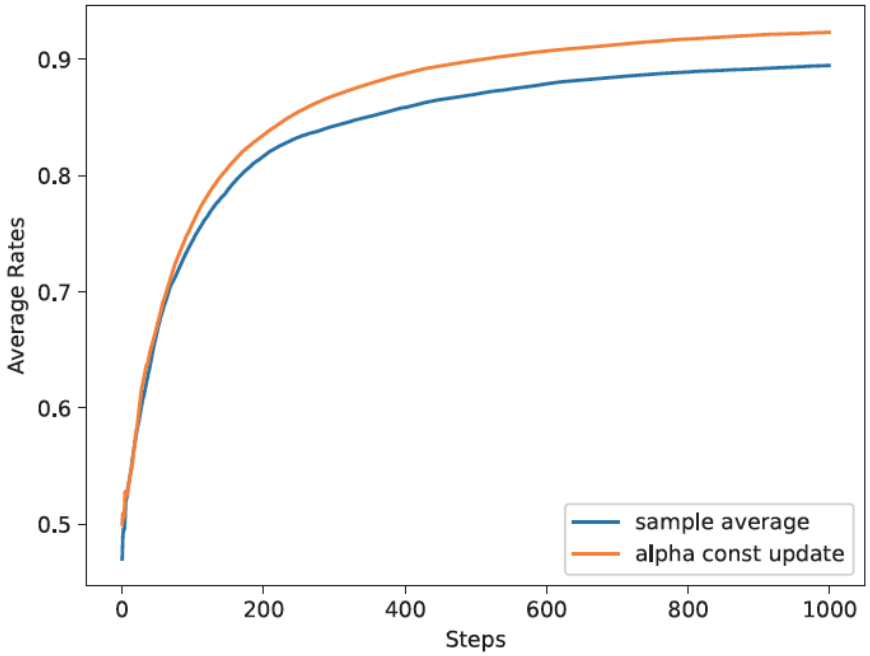
실습 #3 non_stationary.py

```python
import numpy as np
import matplotlib.pyplot as plt
from bandit import Agent


class NonStatBandit:
    def __init__(self, arms=10):
        self.arms = arms
        self.rates = np.random.rand(arms)

    def play(self, arm):
        rate = self.rates[arm]
        self.rates += 0.1 * np.random.randn(self.arms)  # 노이즈 추가
        if rate > np.random.rand():
            return 1
        else:
            return 0


class AlphaAgent:
    def __init__(self, epsilon, alpha, actions=10):
        self.epsilon = epsilon
        self.Qs = np.zeros(actions)
        self.alpha = alpha  # 고정값 α

    def update(self, action, reward):
        # α로 갱신
        self.Qs[action] += (reward - self.Qs[action]) * self.alpha

    def get_action(self):
        if np.random.rand() < self.epsilon:
            return np.random.randint(0, len(self.Qs))
        return np.argmax(self.Qs)
```

```python
runs = 200
steps = 1000
epsilon = 0.1
alpha = 0.8
agent_types = ['sample average', 'alpha const update']
results = {}

for agent_type in agent_types:
    all_rates = np.zeros((runs, steps))  # (200, 1000)

    for run in range(runs):
        if agent_type == 'sample average':
            agent = Agent(epsilon)
        else:
            agent = AlphaAgent(epsilon, alpha)

        bandit = NonStatBandit()
        total_reward = 0
        rates = []

        for step in range(steps):
            action = agent.get_action()
            reward = bandit.play(action)
            agent.update(action, reward)
            total_reward += reward
            rates.append(total_reward / (step + 1))

        all_rates[run] = rates

    avg_rates = np.average(all_rates, axis=0)
    results[agent_type] = avg_rates

# [그림 1-20] 표본 평균과 고정값 α에 의한 갱신 비교
plt.figure()
plt.ylabel('Average Rates')
plt.xlabel('Steps')
for key, avg_rates in results.items():
    plt.plot(avg_rates, label=key)
plt.legend()
plt.show()
```

Implementation

실습 #3



Bandit Problem

- 응용 예

1. 온라인 광고 최적화 (Ad Placement)

- Google, Facebook 등의 온라인 광고 플랫폼에서는 여러 광고 중 가장 효과적인 광고를 찾아야 합니다.
- 특정 광고를 노출할 때마다 클릭률(CTR)을 학습하고, 가장 높은 수익을 내는 광고를 점진적으로 찾아가는 데 Multi-Armed Bandit(MAB) 알고리즘을 사용합니다.

3. 추천 시스템 (Recommendation Systems)

- 유저에게 맞는 콘텐츠를 추천할 때, 처음에는 랜덤하게 추천하면서 유저의 반응(클릭, 구매 등)을 학습.
- 이후 가장 좋은 성과를 내는 추천 알고리즘을 점점 더 활용하는 방식.

5. 로보틱스 및 실시간 제어 시스템

- 로봇이 여러 개의 동작을 실행할 때, 어떤 동작이 가장 효과적인지를 학습하면서 최적의 행동을 선택.
- 예) 로봇 팔이 다양한 방법으로 물건을 집어들면서 최적의 각도를 학습.

7. 주식 및 금융 거래 (Algorithmic Trading)

- 여러 개의 투자 전략(예: 기술적 분석, 뉴스 기반 투자, AI 모델 등) 중에서 최적의 수익을 내는 전략을 찾아가는 과정.
- 특정 주식을 매수할지, 매도할지를 결정할 때 Bandit 알고리즘 활용 가능.